

ANOMALY DETECTION & TIME SERIES

Assignment Code: DA-AG-018

QUESTION 1: What is Anomaly Detection? Explain its types (point, contextual, and collective anomalies) with examples.

Answer:

Anomaly Detection is the process of identifying data points or patterns that significantly differ from normal behavior.

Anomalies are also called **outliers**. They may represent errors, fraud, equipment failures, unusual events, or important changes in a system.

There are three major types of anomalies:

1. Point Anomaly

A **point anomaly** occurs when a single observation is significantly different from the rest of the data.

Example:

Suppose the normal daily electricity consumption of a house is between 10 and 20 kWh, but one day it is 100 kWh.

12, 15, 17, 14, 16, 100, 18, 15

↑
Point Anomaly

The value 100 is an unusual individual observation.

2. Contextual Anomaly

A **contextual anomaly** is a data point that is abnormal only in a particular context.

The context may include:

- Time
- Location
- Season
- Weather
- User behavior

Example:

Electricity consumption of 80 kWh may be normal for a factory during working hours but abnormal for the same factory at midnight.

Therefore, the same value can be normal in one context and anomalous in another.

3. Collective Anomaly

A **collective anomaly** occurs when a group or sequence of observations is unusual, even though individual observations may not appear abnormal.

Example:

Suppose a server normally receives:

100, 105, 110, 98, 103 requests/minute

Suddenly it receives:

150, 155, 160, 158, 162

Each value might not be an extreme outlier individually, but the sequence as a whole may indicate a cyberattack or abnormal traffic pattern.

Comparison:

Type	Meaning	Example
Point Anomaly	One unusual observation	₹10,00,000 transaction

Contextual Anomaly Unusual in a specific context High electricity use at midnight
Collective Anomaly Group/sequence is unusual Sudden sustained server traffic

Real-World Applications:

- Fraud detection
- Network intrusion detection
- Credit card monitoring
- Healthcare monitoring
- Equipment failure detection
- Energy consumption monitoring
- Cybersecurity

Easy Definition:

Anomaly Detection identifies observations or patterns that are significantly different from normal behavior.

QUESTION 2: Compare Isolation Forest, DBSCAN, and Local Outlier Factor in terms of their approach and suitable use cases.

Answer:

Isolation Forest, DBSCAN, and Local Outlier Factor (LOF) are commonly used for detecting unusual observations, but they use different approaches.

1. Isolation Forest

Isolation Forest detects anomalies by randomly selecting features and split values to isolate observations.

Anomalies are generally easier to isolate because they are rare and different from normal observations.

Suitable Use Cases:

- Fraud detection
 - Network attacks
 - Large datasets
 - Sensor anomaly detection
 - Financial transactions
-

2. DBSCAN

DBSCAN is a density-based clustering algorithm.

It identifies dense groups of observations as clusters and points that do not belong to sufficiently dense regions can be classified as noise.

In Scikit-learn, noise points receive the label `-1`.

Suitable Use Cases:

- Spatial data
 - Geographical analysis
 - Irregularly shaped clusters
 - Noise detection
 - Location-based data
-

3. Local Outlier Factor (LOF)

LOF detects anomalies by comparing the **local density** of a point with the local density of its neighbors.

If a point has substantially lower local density than its neighbors, it can be considered an outlier.

Suitable Use Cases:

- Local anomaly detection
- Customer behavior analysis
- Sensor data
- Data with regions having different densities

Comparison Table:

Feature	Isolation Forest	DBSCAN	LOF
Main idea	Isolates unusual points	Finds dense regions	Compares local density
Type	Anomaly detection	Clustering + noise detection	Local outlier detection
Needs number of clusters?	No	No	No
Detects noise?	Yes	Yes	Yes
Good for large datasets	Yes	Can be	Moderate
Handles irregular clusters	Not its main purpose	Yes	Can detect local outliers
Sensitive to density	Less directly	Yes	Yes
Main parameters	<code>n_estimators,</code> <code>contamination</code>	<code>eps, min_samples</code>	<code>n_neighbors,</code> <code>contamination</code>

Easy Definition:

Isolation Forest isolates anomalies, DBSCAN finds low-density noise, and LOF compares the local density of each point with its neighbors.

QUESTION 3: What are the key components of a Time Series? Explain each with one example.

Answer:

A **Time Series** is a sequence of observations recorded in chronological order.

The main components of a time series are:

1. Trend

Trend represents the long-term upward or downward movement of the data.

Example:

A company's yearly sales increase continuously:

2021 → ₹10 lakh

2022 → ₹12 lakh

2023 → ₹15 lakh

2024 → ₹18 lakh

This represents an upward trend.

2. Seasonality

Seasonality represents a pattern that repeats at regular and known intervals.

Example:

Ice cream sales increase every summer and decrease during winter.

Summer → High

Winter → Low

Summer → High

Winter → Low

3. Cyclical Component

A cycle represents longer-term fluctuations that do not necessarily occur at a fixed, known frequency.

Example:

Economic activity may rise and fall over several years due to business cycles.

4. Irregular/Residual Component

This represents random or unexplained variations that remain after modeling systematic components.

Example:

A sudden drop in sales because of an unexpected power outage or natural disaster.

Decomposition:

A time series can often be represented as:

$$Y_t = \text{Trend}_t + \text{Seasonal}_t + \text{Cyclical}_t + \text{Residual}_t$$

For a multiplicative decomposition:

$$Y_t = \text{Trend}_t \times \text{Seasonal}_t \times \text{Cyclical}_t \times \text{Residual}_t$$

Easy Definition:

A time series can contain trend, seasonality, cycles, and random/residual variations.

QUESTION 4: Define Stationary in Time Series. How can you test and transform a non-stationary series into a stationary one?

Answer:

A **stationary time series** has statistical properties that remain reasonably stable over time.

In particular, its:

- Mean
- Variance
- Autocovariance structure

should not systematically change with time.

Example of Non-Stationary Series:

A company's sales continuously increase over time.

100
120
140
160
180
200

The mean is changing, so the series is not stationary.

How to Test Stationarity?

1. Visual Inspection

Plot the time series and check for:

- Trend
- Changing variance
- Seasonality

2. Augmented Dickey-Fuller (ADF) Test

The ADF test is commonly used to test for a unit root.

In the usual interpretation:

- **p-value < 0.05** → reject the unit-root null → evidence supporting stationarity.
- **p-value ≥ 0.05** → insufficient evidence to reject the null.

Python:

```
from statsmodels.tsa.stattools import adfuller
```

```
result = adfuller(series.dropna())
```

```
print("ADF Statistic:", result[0])
```

```
print("p-value:", result[1])
```

How to Transform a Non-Stationary Series?

1. Differencing

Subtract the previous observation:

$$Y'_t = Y_t - Y_{t-1} \quad Y'_t = Y_t - Y_{t-1}$$

Python:

```
series_diff = series.diff().dropna()
```

2. Log Transformation

Useful when the variance increases with the level.

```
import numpy as np
```

```
log_series = np.log(series)
```

3. Seasonal Differencing

For monthly data with yearly seasonality:

$$Y'_t = Y_t - Y_{t-12} \quad Y'_t = Y_t - Y_{t-12}$$

Python:

```
seasonal_diff = series.diff(12).dropna()
```

Easy Definition:

Stationary data has stable statistical behavior over time; differencing and transformations can help make a non-stationary series stationary.

QUESTION 5: Differentiate between AR, MA, ARIMA, SARIMA, and SARIMAX models in terms of structure and application.

Answer:

1. AR – AutoRegressive Model

AR uses previous values of the time series to predict the current value.

An AR(p) model can be written as:

$$Y_t = c + \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \dots + \phi_p Y_{t-p} + \epsilon_t$$
$$Y_t = c + \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \dots + \phi_p Y_{t-p} + \epsilon_t$$

Application:

Useful when current values depend on previous observations.

2. MA – Moving Average Model

MA uses previous forecast errors to predict the current value.

An MA(q) model is:

$$Y_t = c + \epsilon_t + \theta_1 \epsilon_{t-1} + \dots + \theta_q \epsilon_{t-q}$$
$$Y_t = c + \epsilon_t + \theta_1 \epsilon_{t-1} + \dots + \theta_q \epsilon_{t-q}$$

Application:

Useful when current observations are influenced by previous shocks/errors.

3. ARIMA

ARIMA stands for:

AutoRegressive Integrated Moving Average

It combines:

- AR → autoregressive component
- I → differencing
- MA → moving-average error component

Written as:

ARIMA(p,d,q)ARIMA(p,d,q)

Where:

- p = AR order
- d = number of differences
- q = MA order

Application:

Non-seasonal time series forecasting.

4. SARIMA

SARIMA stands for:

Seasonal ARIMA

It adds seasonal AR, differencing, and MA components.

Written as:

SARIMA(p,d,q)(P,D,Q,s)SARIMA(p,d,q)(P,D,Q,s)

Where:

- p,d,q = non-seasonal orders
- P,D,Q = seasonal orders
- s = seasonal period

Application:

Monthly airline passenger forecasting, where yearly seasonality exists.

5. SARIMAX

SARIMAX is SARIMA with **exogenous variables**.

Written conceptually as:

$\text{SARIMAX}(p,d,q)(P,D,Q,s)+X$

Where X represents external explanatory variables.

Example:

Electricity demand can depend on:

- Past electricity demand
- Seasonality
- Temperature
- Humidity
- Holidays

Comparison:

Model	Main Structure	Application
AR	Previous values	Autoregressive series
MA	Previous errors	Error-driven series
ARIMA	AR + Differencing + MA	Non-seasonal forecasting
SARIMA	ARIMA + Seasonality	Seasonal forecasting
SARIMAX	SARIMA + External Variables	Forecasting with external factors

Easy Definition:

AR uses past values, **MA** uses past errors, **ARIMA** adds differencing, **SARIMA** adds seasonality, and **SARIMAX** additionally uses external variables.

QUESTION 6: Load a Time Series Dataset (e.g., AirPassengers), Plot the Original

Series, and Decompose It into Trend, Seasonality, and Residual Components.

Answer:

The AirPassengers dataset contains monthly airline passenger observations and is a classic example of a time series with trend and seasonality.

Google Colab Code:

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

# AirPassengers dataset
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"

df = pd.read_csv(url)

# Convert Month to datetime
df["Month"] = pd.to_datetime(df["Month"])

# Set Month as index
df.set_index("Month", inplace=True)

# Rename column
df.columns = ["Passengers"]

# Plot original series
plt.figure(figsize=(12, 5))

plt.plot(
    df["Passengers"]
)

plt.title("AirPassengers Time Series")
plt.xlabel("Date")
plt.ylabel("Number of Passengers")

plt.show()

# Time series decomposition
```

```
decomposition = seasonal_decompose(  
    df["Passengers"],  
    model="multiplicative",  
    period=12  
)  
  
# Plot decomposition  
fig = decomposition.plot()  
  
fig.set_size_inches(  
    12,  
    9  
)  
  
plt.show()
```

Expected Output:

The decomposition produces four components:

1. Observed
2. Trend
3. Seasonal
4. Residual

Interpretation:

- **Observed:** Original passenger series.
- **Trend:** Long-term increase in passenger numbers.
- **Seasonal:** Repeating yearly pattern.
- **Residual:** Remaining unexplained variation.

Observation:

The AirPassengers series shows a strong upward trend along with repeating seasonal behavior.

QUESTION 7: Apply Isolation Forest on a Numerical Dataset (e.g., NYC Taxi Fare) to

Detect Anomalies. Visualize the Anomalies on a 2D Scatter Plot.

Answer:

Isolation Forest is an unsupervised anomaly detection algorithm that isolates unusual observations through random partitioning.

For a numerical taxi dataset, features such as:

- Fare amount
- Distance
- Passenger count

can be used.

Google Colab Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.ensemble import IsolationForest

# Create sample taxi-like numerical data
np.random.seed(42)

normal_data = np.column_stack([
    np.random.uniform(2, 30, 500),    # Fare
    np.random.uniform(1, 20, 500)     # Distance
])

# Add artificial anomalies
anomalies = np.array([
    [150, 2],
    [200, 3],
    [180, 1],
    [5, 60],
    [8, 80],
    [250, 5]
])
```

```

X = np.vstack([
    normal_data,
    anomalies
])

taxi_df = pd.DataFrame(
    X,
    columns=[
        "fare_amount",
        "distance"
    ]
)

# Isolation Forest
model = IsolationForest(
    contamination=0.02,
    random_state=42
)

taxi_df["anomaly"] = model.fit_predict(
    taxi_df[
        ["fare_amount", "distance"]
    ]
)

# -1 = anomaly
# 1 = normal

# Plot
plt.figure(figsize=(10, 6))

normal = taxi_df[
    taxi_df["anomaly"] == 1
]

outliers = taxi_df[
    taxi_df["anomaly"] == -1
]

plt.scatter(
    normal["fare_amount"],
    normal["distance"],
    s=25,
    label="Normal"

```

```
)

plt.scatter(
    outliers["fare_amount"],
    outliers["distance"],
    marker="x",
    s=100,
    label="Anomaly"
)

plt.xlabel("Fare Amount")
plt.ylabel("Distance")
plt.title("Isolation Forest Anomaly Detection")
plt.legend()

plt.show()

print(
    "Number of anomalies:",
    len(outliers)
)
```

Output:

The scatter plot displays:

- Normal observations as regular points.
- Anomalies as **X** markers.

Observation:

Isolation Forest identifies observations that are unusual compared with the majority of the numerical data.

Note: The code above uses synthetic taxi-like data so that it runs directly in Colab. If an actual NYC Taxi Fare CSV is available, replace the synthetic-data section with the real dataset and select appropriate numerical columns.

QUESTION 8: Train a SARIMA Model on the Monthly Airline Passengers Dataset. Forecast the Next 12 Months and Visualize the Results.

Answer:

SARIMA is suitable for the AirPassengers dataset because the data contains strong yearly seasonality.

For monthly data:

Seasonal period = 12

Google Colab Code:

```
import pandas as pd
import matplotlib.pyplot as plt

from statsmodels.tsa.statespace.sarimax import SARIMAX

# Load AirPassengers
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"

df = pd.read_csv(url)

# Convert date
df["Month"] = pd.to_datetime(
    df["Month"]
)

df.set_index(
    "Month",
    inplace=True
)

df.columns = ["Passengers"]

# Build SARIMA model
```

```

model = SARIMAX(
    df["Passengers"],
    order=(1, 1, 1),
    seasonal_order=(1, 1, 1, 12),
    enforce_stationarity=False,
    enforce_invertibility=False
)

# Train
model_fit = model.fit(
    disp=False
)

# Forecast next 12 months
forecast = model_fit.forecast(
    steps=12
)

# Print forecast
print("Next 12 Months Forecast:")
print(forecast)

# Plot
plt.figure(figsize=(12, 6))

plt.plot(
    df.index,
    df["Passengers"],
    label="Historical"
)

plt.plot(
    forecast.index,
    forecast,
    marker="o",
    label="Forecast"
)

plt.xlabel("Date")
plt.ylabel("Passengers")
plt.title(
    "SARIMA Forecast - Next 12 Months"
)

```

```
plt.legend()  
plt.show()
```

Expected Output:

The program prints 12 future monthly passenger forecasts and displays them on a graph.

Observation:

The SARIMA model captures:

- Long-term trend
- Seasonal yearly pattern
- Short-term time-series dependence

and produces forecasts for the next 12 months.

QUESTION 9: Apply Local Outlier Factor (LOF) on Any Numerical Dataset to Detect Anomalies and Visualize Them Using Matplotlib.

Answer:

Local Outlier Factor (LOF) identifies observations whose local density is substantially lower than that of their neighbors.

Google Colab Code:

```
import numpy as np  
import matplotlib.pyplot as plt  
  
from sklearn.neighbors import LocalOutlierFactor  
  
# Generate normal data  
np.random.seed(42)
```

```
normal_data = np.random.normal(  
    loc=0,  
    scale=1,  
    size=(500, 2)  
)
```

```
# Generate anomalies
```

```
outliers = np.array([  
    [6, 6],  
    [7, 5],  
    [-6, -5],  
    [-7, -6],  
    [8, -7]  
)
```

```
# Combine
```

```
X = np.vstack([  
    normal_data,  
    outliers  
)
```

```
# LOF
```

```
model = LocalOutlierFactor(  
    n_neighbors=20,  
    contamination=0.02  
)
```

```
labels = model.fit_predict(  
    X  
)
```

```
# -1 = anomaly
```

```
# 1 = normal
```

```
normal = X[  
    labels == 1  
)
```

```
anomalies = X[  
    labels == -1  
)
```

```
# Visualization
```

```
plt.figure(figsize=(9, 6))
```

```
plt.scatter(
    normal[:, 0],
    normal[:, 1],
    s=25,
    label="Normal"
)

plt.scatter(
    anomalies[:, 0],
    anomalies[:, 1],
    marker="x",
    s=100,
    label="Anomaly"
)

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Local Outlier Factor Anomaly Detection")
plt.legend()

plt.show()

print(
    "Number of anomalies:",
    len(anomalies)
)
```

Output:

The graph highlights the observations classified as anomalies.

Observation:

LOF is useful when an observation is unusual relative to its **local neighborhood**, rather than only being globally far away from the rest of the data.

QUESTION 10: Power Grid Monitoring – Real-Time Energy Demand Forecasting and Anomaly Detection

Answer:

A power-grid monitoring company wants to forecast energy demand and detect abnormal spikes or drops in real-time consumption data collected every 15 minutes.

The dataset contains:

- Timestamp
- Region
- Weather conditions
- Energy usage

The complete workflow should combine **data preprocessing, anomaly detection, forecasting, validation, monitoring, and business actions**.

Step 1: Collect Real-Time Data

The system receives new observations every 15 minutes.

Example:

Timestamp	Region	Temperature	Energy
10:00	North	30°C	500 MW
10:15	North	31°C	510 MW
10:30	North	31°C	515 MW
10:45	North	30°C	1200 MW

The 1200 MW observation may represent an abnormal spike.

Step 2: Data Preprocessing

First:

- Convert timestamp into datetime.
- Sort observations by timestamp.
- Check duplicate records.
- Handle missing values.
- Validate sensor readings.
- Handle invalid values.
- Resample if necessary.
- Create useful time-based features.

Example Features:

Hour

Day of Week

Month

Weekend

Temperature

Humidity

Previous Energy Usage

Python:

```
import pandas as pd
```

```
df["timestamp"] = pd.to_datetime(  
    df["timestamp"]  
)
```

```
df = df.sort_values(  
    "timestamp"  
)
```

```
df["hour"] = df[  
    "timestamp"  
].dt.hour
```

```
df["day_of_week"] = df[  
    "timestamp"  
].dt.dayofweek
```

```
df["month"] = df[  
    "timestamp"  
].dt.month
```

```
df["is_weekend"] = (  
    df["day_of_week"] >= 5  
)<div data-bbox="111 214 808 281" data-label="Section-Header">

---



## Step 3: Detect Anomalies in Streaming Data


```

Three possible algorithms are:

Isolation Forest

Best choice when:

- Dataset is large.
- Features are numerical.
- Fast anomaly detection is required.
- Data has potentially complex anomaly patterns.

LOF

Useful when:

- Local density matters.
- Anomalies are unusual relative to nearby observations.

DBSCAN

Useful when:

- Data has meaningful density structure.
- We want to identify low-density observations as noise.

Recommended Approach

For a real-time power grid system, I would primarily use **Isolation Forest** because it is suitable for numerical, potentially high-volume data and can be used efficiently in an operational anomaly-detection pipeline.

Python Example:

```
from sklearn.ensemble import IsolationForest
```

```
features = [  
    "energy_usage",  
    "temperature",  
    "humidity"  
]  
  
model = IsolationForest(  
    n_estimators=100,  
    contamination=0.01,  
    random_state=42  
)  
  
df["anomaly"] = model.fit_predict(  
    df[features]  
)  
  
# -1 = anomaly  
# 1 = normal  
  
anomalies = df[  
    df["anomaly"] == -1  
]  
  
print(  
    "Detected anomalies:",  
    len(anomalies)  
)
```

Step 4: Create Time-Series Features

Because electricity demand is affected by time, create lag and rolling features.

Lag Feature:

```
df["lag_1"] = df[  
    "energy_usage"  
].shift(1)
```

For 15-minute observations:

lag_1 = previous 15 minutes

lag_4 = previous 1 hour

lag_96 = previous 24 hours

Rolling Mean:

```
df["rolling_mean"] = (  
    df["energy_usage"]  
    .rolling(4)  
    .mean()  
)
```

Step 5: Choose the Forecasting Model

The choice between ARIMA, SARIMA, and SARIMAX depends on the data.

ARIMA

Use ARIMA when:

- The series is primarily non-seasonal.
 - External variables are not needed.
-

SARIMA

Use SARIMA when:

- Strong seasonality exists.
- Historical demand itself is the main source of predictive information.

For 15-minute electricity demand, there may be:

- Daily seasonality
- Weekly seasonality

A standard SARIMA model can become cumbersome when there are multiple seasonal periods.

SARIMAX

I would prefer **SARIMAX** when weather variables and other external factors are important.

For example:

Energy Demand



Temperature

Humidity

Hour

Day of Week

Holiday



SARIMAX



Forecast

Why SARIMAX?

Electricity demand is influenced not only by previous demand but also by:

- Temperature
- Humidity
- Weather
- Time of day
- Day of week
- Holidays

Therefore, SARIMAX can incorporate external variables.

Step 6: Example SARIMAX Code

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
```

```
# Prepare data
```

```
data = df.dropna().copy()
```

```

y = data[
    "energy_usage"
]

exog = data[
    [
        "temperature",
        "humidity"
    ]
]

# Train-test split
split = int(
    len(data) * 0.8
)

y_train = y.iloc[:split]
y_test = y.iloc[split:]

exog_train = exog.iloc[:split]
exog_test = exog.iloc[split:]

# SARIMAX model
model = SARIMAX(
    y_train,
    exog=exog_train,
    order=(1, 0, 1),
    seasonal_order=(1, 1, 1, 96),
    enforce_stationarity=False,
    enforce_invertibility=False
)

model_fit = model.fit(
    disp=False
)

# Forecast
forecast = model_fit.forecast(
    steps=len(y_test),
    exog=exog_test
)

print(forecast.head())

```

Important:

For 15-minute data, 96 observations represent one day:

$$24 \times 4 = 96$$

The exact seasonal structure should be determined from the actual data rather than assumed blindly.

Step 7: Evaluate Forecasting Performance

Use metrics such as:

MAE

$$\text{MAE} = \frac{1}{n} \sum |y_i - \hat{y}_i| \quad \text{MAE} = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

RMSE

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2} \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$$

MAPE

$$\text{MAPE} = \frac{100}{n} \sum \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad \text{MAPE} = \frac{100}{n} \sum \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Python:

```
from sklearn.metrics import (  
    mean_absolute_error,  
    mean_squared_error  
)
```

```
import numpy as np
```

```
mae = mean_absolute_error(  
    y_test,  
    forecast  
)
```

```
rmse = np.sqrt(  
    mean_squared_error(  
        y_test,  
        forecast  
    )  
)
```

```
        y_test,  
        forecast  
    )  
)  
  
print("MAE:", mae)  
print("RMSE:", rmse)
```

Step 8: Evaluate Anomaly Detection

For anomaly detection, evaluation depends on whether labeled anomaly data exists.

If labels are available, use:

- Precision
- Recall
- F1-Score
- ROC-AUC
- PR-AUC

If labels are unavailable, use:

- Expert validation
 - Historical incidents
 - Alert investigation results
 - False-alert rate
 - Stability of anomaly scores
-

Step 9: Real-Time Monitoring

The system should continuously monitor:

Forecast Performance

- MAE
- RMSE
- Forecast bias

Anomaly Detection

- Number of anomalies
- Anomaly rate
- False alerts
- Missed events

Data Quality

- Missing values
- Delayed data
- Invalid sensor readings
- Sudden distribution changes

Model Drift

Monitor whether the relationship between inputs and demand changes over time.

Step 10: Real-Time Alert System

If an abnormal spike or drop is detected:

Incoming Data



Data Validation



Feature Generation



Anomaly Detection



Forecasting



Compare Actual vs Expected



Anomaly?



YES



NO



Alert



Continue

Example:

```
if anomaly_detected:
    print(
        "ALERT: Abnormal energy consumption detected!"
    )
```

Step 11: Business Value

This system can provide significant value to a power-grid monitoring company.

1. Prevent Equipment Failure

Unusual energy patterns can indicate equipment problems.

2. Improve Grid Stability

Early detection of abnormal demand helps operators respond before the situation becomes critical.

3. Better Demand Planning

Accurate forecasts help determine expected future electricity demand.

4. Reduce Operational Costs

Better forecasting helps optimize:

- Power generation
- Energy purchasing
- Storage
- Grid resources

5. Detect Abnormal Consumption

The system can identify unusual consumption patterns that may require investigation.

6. Improve Maintenance

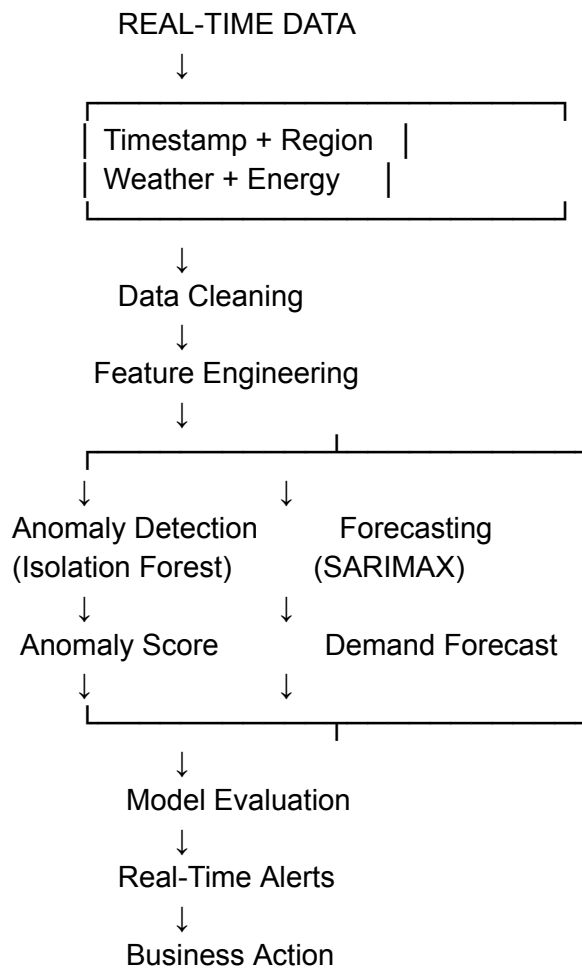
Anomaly patterns can help prioritize equipment inspection and preventive maintenance.

7. Support Real-Time Decision Making

Operators can receive alerts when:

- Demand suddenly spikes.
 - Demand unexpectedly drops.
 - Forecast error becomes unusually large.
 - Sensors behave abnormally.
-

COMPLETE POWER GRID WORKFLOW



↓
Continuous Monitoring

FINAL COMPARISON

Concept	Main Purpose
Anomaly Detection	Find unusual observations
Point Anomaly	Individual unusual point
Contextual Anomaly	Unusual within a particular context
Collective Anomaly	Unusual group/sequence
Isolation Forest	Isolate unusual observations
DBSCAN	Density-based clustering/noise detection
LOF	Detect local density-based outliers
Time Series	Chronologically ordered observations
Trend	Long-term direction
Seasonality	Repeating pattern
Cycle	Longer-term fluctuation
Residual	Unexplained variation
AR	Uses previous values
MA	Uses previous errors
ARIMA	AR + Differencing + MA
SARIMA	ARIMA + Seasonality
SARIMAX	SARIMA + External Variables
ADF Test	Tests for unit-root/non-stationarity
Differencing	Common method to remove trend

MAE	Average absolute forecast error
RMSE	Penalizes larger errors more
MAPE	Percentage-based forecast error

IMPORTANT FORMULAS

MAE

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

RMSE

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Differencing

$$Y'_t = Y_t - Y_{t-1}$$

Seasonal Differencing

$$Y'_t = Y_t - Y_{t-s}$$

Silhouette Score is NOT an anomaly metric

It is primarily a **clustering evaluation metric**, while Isolation Forest and LOF are anomaly-detection methods.

CONCLUSION

Anomaly Detection and Time Series Analysis are important techniques for finding unusual behavior and forecasting future values.

Isolation Forest, **DBSCAN**, and **LOF** provide different approaches for identifying anomalies. Isolation Forest isolates unusual observations, DBSCAN uses density-based clustering and noise detection, while LOF compares the local density of observations.

For time-series forecasting, **ARIMA** is useful for non-seasonal series, **SARIMA** is useful when seasonality is present, and **SARIMAX** is useful when external variables such as weather also influence the target.

In a real-world power-grid system, combining real-time anomaly detection with SARIMAX-based forecasting can help detect abnormal energy consumption, improve demand planning, reduce operational risks, and support faster business decisions.